

Servicio de IA: Análisis, Diseño y Especificación Técnica Completa

Estudio de ingeniería de software, requerimientos funcionales y no funcionales, análisis de compensaciones tecnológicas (Trade-offs), arquitectura analítica ONNX Runtime y matriz formal de casos de uso.

1. Descripción General

El **Servicio de IA** es un microservicio crítico de alta disponibilidad y baja latencia diseñado bajo el framework FastAPI. Su propósito fundamental es la clasificación automatizada en tiempo real de residuos sólidos introducidos en los contenedores ecológicos inteligentes EcoSmartBin.

A diferencia de los despliegues tradicionales basados en pesados entornos como TensorFlow o PyTorch, este componente ejecuta un modelo matemático optimizado en formato **ONNX (Open Neural Network Exchange)**. Esto garantiza un consumo de recursos de memoria sumamente estricto, ideal para infraestructuras autoescalables y entornos Serverless.

Características Principales del Sistema

- **Clasificación en tiempo real:** Capacidad nativa para segregar de forma automatizada 6 categorías de residuos urbanos comunes.
- **Arquitectura Ultra-ligera:** Modelo ONNX empaquetado de aproximadamente 100MB optimizado para tiempos de arranque en frío mínimos (Warm Starts acelerados).
- **Mecanismo Síncrono-Asíncrono (Fire-and-forget):** Responde inmediatamente al dispositivo de captura de imágenes (ESP32-CAM) y gestiona la sincronización de datos con el ecosistema transaccional de puntos en un hilo de ejecución secundario no bloqueante.
- **Soporte Multiplataforma Nativo:** Contenedorizado de manera agnóstica preparado para despliegues eficientes en entornos productivos locales o nubes elásticas (Google Cloud Run, AWS Fargate).
- **Análisis Multiclase Avanzado:** El pipeline de inferencia no solo retorna la clase ganadora, sino el Top 3 de predicciones con sus respectivos índices de confianza probabilística (Confidence Scores).

2. Diseño y Análisis del Módulo de IA

El diseño de este módulo responde a rigurosos principios de ingeniería de software orientados a sistemas empotrados y de alta concurrencia. A continuación, se detallan los requerimientos analizados, las decisiones arquitectónicas clave y el análisis de ventajas frente a soluciones alternativas.

2.1. Análisis de Requerimientos

Requerimientos Funcionales (RF)

- **RF-1 Procesamiento de Imagen Base:** El sistema debe recibir flujos de imágenes binarias transmitidos mediante el protocolo HTTP Multipart.
- **RF-2 Pipeline de Transformación Numérica:** El módulo debe transformar cualquier dimensión de imagen de entrada al formato normalizado exigido por el modelo analítico (224x224 píxeles, canales RGB, estandarización ImageNet).
- **RF-3 Inferencia Multiclase Extensible:** El motor analítico debe clasificar la entrada dentro de las 6 categorías de residuos predefinidas.
- **RF-4 Extracción de Distribución Probabilística:** Las respuestas deben contener no solo el elemento de máxima certidumbre (Argmax), sino las 3 mejores alternativas ordenadas (Top-3) junto con su puntuación de confianza (Confidence Scores).
- **RF-5 Sincronización Transaccional en Background:** Al finalizar la inferencia, los metadatos y la imagen codificada en formato legible (Base64) deben ser delegados de manera no bloqueante al Servicio de Puntos.

Requerimientos No Funcionales (RNF)

- **RNF-1 Eficiencia y Baja Latencia:** El tiempo neto de cómputo del pipeline de inferencia interna no debe superar los 150 milisegundos bajo condiciones nominales en CPU de un solo núcleo.
- **RNF-2 Huella Mínima de Memoria (Resource Constrained):** El contenedor de ejecución en reposo no debe consumir más de 200MB de memoria RAM y bajo carga máxima operativa de inferencia continua debe mantenerse estrictamente por debajo de 1GB de RAM.
- **RNF-3 Concurrencia No Bloqueante:** La comunicación síncrona con el hardware periférico (ESP32-CAM) jamás debe verse retrasada o acoplada temporalmente por la disponibilidad o latencias de red de la base de datos o el Servicio de Puntos.
- **RNF-4 Portabilidad y Aislamiento Operativo:** El microservicio debe operar de forma idéntica en hardware local de desarrollo Linux/Mac/Windows mediante capas de abstracción Docker y soportar el escalado automático horizontal a cero en la nube (Cloud Run).

2.2. Análisis de Compensaciones Tecnológicas (Trade-off Analysis)

Para fundamentar el diseño técnico, se evaluaron críticamente las decisiones arquitectónicas frente a los enfoques tradicionales de la industria:

Dimensión del Diseño	Enfoque Elegido: ONNX Runtime + FastAPI Async	Alternativa Evaluada: TensorFlow / PyTorch Nativo	Justificación de Ingeniería
Motor de Inferencia Analítica	<code>onnxruntime</code> (~100MB distribuidos, optimización nativa de grafos de operadores c++).	<code>tensorflow</code> / <code>torch</code> (>1.2GB de tamaño de imagen Docker, alto consumo inercial de RAM).	El uso de ONNX reduce la imagen de despliegue en un 90% y mitiga el problema de arranque en frío (Cold Start) de microservicios serverless, bajando el tiempo inicial de carga de segundos a milisegundos.
Estrategia de Sincronización	Asíncrona (Fire-and-Forget): Delegación mediante <code>asyncio.create_task</code> a hilos de procesamiento de eventos en segundo plano.	Síncrona (Orquestación en Cascada): Esperar el almacenamiento exitoso en la BD corporativa antes de responder al hardware.	El microcontrolador del contenedor físico (ESP32-CAM) tiene recursos extremadamente limitados de energía y timeouts de socket estrictos. Responder síncronamente degradaría la experiencia de usuario y saturaría las conexiones WiFi del bin por bloqueos de red externos.
Transferencia de Archivos Gráficos	Procesamiento puramente en memoria virtual mediante objetos <code>io.BytesIO</code> y conversión intermedia a Base64 para el frontend.	Persistencia temporal de archivos en disco del servidor o almacenamiento de objetos Blob (Google Cloud Storage / AWS S3).	Elimina por completo las latencias críticas de I/O en disco (operación costosa en entornos virtuales) y simplifica el gobierno de datos al evitar la gestión persistente de imágenes residuales temporales, garantizando privacidad por diseño.

3. Arquitectura del Sistema

El diseño de software adopta un enfoque desacoplado, dividiendo las responsabilidades de captura de hardware, inferencia analítica y procesamiento de lógicas de negocio/gamificación en microservicios independientes conectados por HTTP estructurado.

Flujo Completo del Ciclo de Vida del Dato



Diagrama de Capas del Microservicio

```
| FastAPI REST Framework (Manejo de Routing, Validaciones y Lifespan) |
| Preprocessing Layer (Normalización Imagenet, Redimensionamiento 224x224) |
| ONNX Runtime Session Management Engine (Carga única de garbage_model.onnx) |
| Post-processing Layer (Aplicación de Softmax, Argmax y Extracción Top-3) |
| Async Background Sync Layer (Pool reusable httpx.AsyncClient) |
```

4. Componentes Principales

4.1. Modelo ONNX (`garbage_model.onnx`)

Convertido y optimizado desde arquitecturas complejas de Deep Learning, sus especificaciones de ejecución matemática contemplan un tensor de entrada estricto de dimensiones `(1, 3, 224, 224)` en formato de canal flotante de 32 bits (float32). Sus salidas corresponden a los Logits crudos de las 6 clases soportadas por el sistema.

4.2. Mapeo de Categorías (`labels.json`)

Archivo de configuración JSON estático encargado de mapear las posiciones indexadas del array de salida del modelo con las etiquetas semánticas de los residuos comerciales:

```
{
  "0": "cardboard",
  "1": "glass",
  "2": "metal",
  "3": "paper",
  "4": "plastic",
  "5": "trash"
}
```

4.3. Inicialización Eficiente mediante Lifespan

Para evitar la sobrecarga masiva de operaciones de Entrada/Salida (I/O) en disco que penalizan la ejecución por cada petición entrante, el modelo analítico se carga en memoria RAM de forma única durante la inicialización del contenedor, permaneciendo residente durante todo el ciclo de ejecución del servicio:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Setup de arranque (Startup)
    # - Inicialización de la InferenceSession de ONNX Runtime
    # - Carga de labels de mapeo semántico desde disco
    # - Instanciación del Pool global de conexiones HTTP asíncronas
    yield
    # Lógica de apagado limpio (Shutdown)
    # - Cierre controlado del pool de conexiones http_client
    # - Liberación explícita de estructuras de datos en memoria
```

5. Flujo de Procesamiento Detallado

El ciclo analítico y operacional del sistema comprende una serie ordenada de transformaciones del dato y llamadas de comunicación de red estructuradas de la siguiente manera:

1. **Captura en Hardware (ESP32-CAM):** Al accionarse el mecanismo del contenedor, el chip SoC activa el sensor OV2640, capturando la toma gráfica en un buffer JPEG optimizado en PSRAM (Resolución VGA 640x480 con factor de calidad balanceado).
2. **Transmisión Multipart sobre WiFi:** Se inicializa una petición HTTP POST binaria hacia el microservicio codificada bajo el protocolo estándar `multipart/form-data` usando límites definidos (Boundaries).
3. **Preprocesamiento Numérico de la Imagen:** Al recibir los bytes del flujo de red, la API decodifica la imagen sin volcarla a disco (operación puramente en memoria virtual) aplicando las siguientes operaciones de álgebra lineal:
 - Redimensionamiento bilineal exacto a 224x224 píxeles.
 - Normalización escalar de la matriz de enteros [0-255] a flotantes con rango [0-1].
 - Estandarización basada en pesos estadísticos ImageNet utilizando medias (`MEAN = [0.485, 0.456, 0.406]`) y desviaciones estándar (`STD = [0.229, 0.224, 0.225]`).
 - Transposición estructural de ejes pasando de formato HWC (Height, Width, Channels) a CHW (Channels, Height, Width) requerida por el estándar ONNX.
 - Expansión de dimensiones para conformar el lote batch de inferencia `(1, 3, 224, 224)` .
4. **Inferencia Matemática:** Los datos se inyectan a la sesión activa de ejecución de `onnxruntime` mediante el método `run()` , aprovechando las optimizaciones de grafos unificados del procesador físico.
5. **Postprocesamiento Estadístico:** Los Logits crudos resultantes de la red neuronal se transforman a través de una función Softmax numéricamente estable para derivar probabilidades de distribución percentil. Se ejecuta un ordenamiento inverso (Argmax) para extraer el Top 3 de clasificaciones candidatas acompañadas de su certidumbre matemática.
6. **Notificación en Segundo Plano (Fire-and-Forget):** Mediante el gestor de eventos asíncronos de Python (`asyncio.create_task`), el servicio delega una petición HTTP paralela al microservicio de puntos

conteniendo la etiqueta, ID del bin e imagen codificada en formato Base64 estándar, liberando inmediatamente al hardware físico.

6. API REST Endpoints Reference

6.1. Inferencia Analítica Principal

Endpoint: `POST /predict`

Content-Type: `multipart/form-data`

Estructura Básica del Cuerpo de Solicitud (HTTP Request):

```
POST /predict HTTP/1.1
Host: servicio-ia.run.app
Content-Type: multipart/form-data; boundary=----BoundaryEx

-----BoundaryEx
Content-Disposition: form-data; name="file"; filename="residuo.jpg"
Content-Type: image/jpeg

[DATOS_BINARIOS_RAW_JPEG]
-----BoundaryEx--
```

Estructura de Respuesta Exitosa (200 OK):

```
{
  "success": true,
  "class": "plastic",
  "confidence": 0.9234,
  "top_predictions": [
    { "label": "plastic", "confidence": 0.9234 },
    { "label": "trash", "confidence": 0.0512 },
    { "label": "paper", "confidence": 0.0254 }
  ]
}
```

6.2. Verificación de Estado (Health Check)

Endpoint: `GET /`

Respuesta (200 OK):

```
{
  "status": "healthy",
  "engine": "pure-onnxruntime"
}
```


7. Clasificaciones Soportadas y Matriz Transaccional

A continuación se tabula el comportamiento estricto del motor predictivo y el peso transaccional configurado en el núcleo de negocio para la acreditación de la recompensa ecológica:

ID Clase	Etiqueta Core (IA)	Mapeo Semántico (BD)	Descripción Operacional	Puntos / Unidad
0	cardboard	Cartón	Cajas corrugadas, empaques limpios y cartulinas secas.	3 puntos
1	glass	Vidrio	Botellas de bebidas, frascos alimenticios e insumos vítreos.	5 puntos
2	metal	Metal	Latas de aluminio, contenedores de conserva y hojalata.	8 puntos
3	paper	Papel	Hojas de oficina, periódicos, revistas y folletos.	2 puntos
4	plastic	Plástico	Envases PET, botellas de polietileno de alta densidad (HDPE).	4 puntos
5	trash	Basura General	Residuos sanitarios, orgánicos degradados o no reciclables.	0 puntos

8. Casos de Uso del Sistema

Para comprender la interacción del microservicio con los diferentes componentes del ecosistema de hardware, lógica empresarial y usuarios finales, se definen formalmente los siguientes casos de uso analíticos y operacionales.

Caso de Uso UC-01: Clasificación Automatizada e Inferencia de Residuos

Actores	Contenedor Inteligente (ESP32-CAM), Servicio de IA (Sistema)
Precondiciones	El microservicio de IA se encuentra inicializado con la sesión ONNX cargada en memoria RAM de forma exitosa. El dispositivo ESP32-CAM cuenta con conectividad a la red inalámbrica local.
Flujo Principal (F.P.)	1. El usuario introduce físicamente un material residual y acciona el botón del contenedor EcoSmartBin. 2. El ESP32-CAM captura la fotografía analógica a través del lente óptico y codifica la información en un formato binario estructurado. 3. El hardware emite un request POST HTTP binario hacia el endpoint <code>/predict</code> del Servicio de IA. 4. El Servicio de IA intercepta los bytes en memoria virtual, aplicando las directrices de normalización y transposición de la capa de preprocesamiento. 5. Se ejecuta el grafo analítico de inferencia matemática mediante el motor de ONNX Runtime. 6. El backend calcula la distribución probabilística y retorna de forma inmediata un código de estado HTTP 200 con la predicción ganadora y el listado complementario Top-3 al hardware cliente.
Flujos Alternativos / Excepciones	3a. Formato de Archivo Inválido: Si el archivo enviado no cumple las características de una estructura MIME de imagen soportada, el servicio cancela el flujo y retorna una excepción estructurada <code>HTTP 400 Bad Request</code>. 5a. Falla Crítica de Inferencia: Si ocurre una degradación del tensor matemático o un fallo interno del motor de ejecución, el sistema captura el error de forma controlada emitiendo una alerta en logs internos y respondiendo un código <code>HTTP 500 Internal Server Error</code>.
Postcondiciones	El hardware del contenedor físico conoce con precisión la naturaleza del material classified para proceder con lógicas internas locales y la transmisión no bloqueante se orquesta en paralelo.

Caso de Uso UC-02: Sincronización Asíncrona Transaccional (Fire-and-Forget)

Actores	Servicio de IA (Sistema), Servicio de Puntos (Sistema)
Precondiciones	Se ha completado con éxito la fase de inferencia matemática del Caso de Uso UC-01 y se dispone de una etiqueta predictiva con una certidumbre superior al umbral mínimo del sistema.
Flujo Principal (F.P.)	1. El Servicio de IA, de manera concurrente al envío de la respuesta al cliente físico, abstrae la imagen original convirtiéndola a una cadena serializada codificada en Base64. 2. El sistema engloba los metadatos correspondientes (<code>binId</code> , <code>tipoDetectado</code> , <code>confianza</code> e <code>imagenBase64</code>) dentro de un objeto payload estructurado. 3. Mediante el pool de conexiones optimizado, el servicio dispara una tarea en segundo plano no bloqueante (Background Task) hacia el endpoint <code>/points/clasificacion-pendiente</code> del Servicio de Puntos corporativo. 4. El Servicio de Puntos recibe el objeto y lo aloja temporalmente dentro de su mapa de concurrencia en memoria de alta velocidad (ConcurrentHashMap) indexado bajo la llave única del Identificador de Contenedor (<code>binId</code>).
Flujos Alternativos / Excepciones	3a. Caída de Red / Indisponibilidad del Servicio de Puntos: Si el microservicio transaccional remoto experimenta timeout (límite superior de 5.0 segundos) o rechazo de conexión por indisponibilidad física de infraestructura, la tarea en background captura la excepción en logs sin afectar o degradar el tiempo de respuesta final otorgado al dispositivo ESP32 en el flujo raíz.
Postcondiciones	El ecosistema transaccional mantiene constancia de un proceso de reciclaje pendiente de ratificación por el usuario de manera segura y sin penalizaciones en la experiencia física del hardware.

Caso de Uso UC-03: Ratificación de Clasificación y Adjudicación de Eco-Puntos

Actores	Usuario Final, Aplicación Frontend (Móvil/Web), Servicio de Puntos, Base de Datos
Precondiciones	Existe un registro activo y mapeado en la estructura de almacenamiento volátil del Servicio de Puntos correspondiente al identificador físico del bin consultado.
Flujo Principal (F.P.)	1. El Usuario abre la aplicación e introduce el identificador del contenedor inteligente o escanea el código QR del mismo. 2. La interfaz de usuario ejecuta llamadas iterativas o consultas directas HTTP GET al endpoint del Servicio de Puntos obteniendo la previsualización de la toma física y el mapeo traducido en idioma español. 3. La aplicación renderiza de manera íntegra la imagen decodificada de la cadena Base64 para inspección visual del usuario. 4. El usuario presiona el botón interactivo de "Confirmar Clasificación" en la pantalla de su dispositivo móvil. 5. El Frontend realiza un POST transaccional hacia las API de negocio confirmando el procesamiento de las unidades detectadas. 6. El Servicio de Puntos calcula los valores correspondientes multiplicando las unidades validadas por el peso de la matriz transaccional oficial. 7. Se ejecuta una transacción ACID en el motor persistente actualizando el balance del usuario final y se purga el registro temporal de memoria intermedia mediante una petición DELETE.
Flujos Alternativos / Excepciones	2a. Ausencia de Datos Pendientes (204 No Content): Si el usuario ingresa de forma anticipada antes de que el proceso en background concluya o la imagen sea procesada, la interfaz detecta el estado vacío informando al cliente mediante un mensaje descriptivo de espera. 4a. Discrepancia de Clasificación (Rechazo): Si el usuario identifica que el material fue erróneamente clasificado, puede abortar o seleccionar manualmente la corrección, forzando la limpieza y auditoría del modelo de aprendizaje para procesos futuros.
Postcondiciones	El balance contable ecológico del usuario ha sido incrementado de forma consistente y la memoria RAM transaccional intermedia queda completamente libre de remanentes de datos procesados.

9. Configuración de Despliegue (Deployment)

Para asegurar la máxima reproducibilidad del entorno de software en infraestructuras locales y productivas de nivel corporativo, el microservicio se encuentra estrictamente empaquetado mediante contenedores Docker utilizando compilaciones optimizadas slim de Linux.

Dockerfile de Producción Optimizado

```
FROM python:3.12-slim

WORKDIR /app

# Instalación obligatoria de libgomp1 para soportar la paralelización OpenMP de ONNX Runtime
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

Nota Crítica de Infraestructura: Dependencia de Linux libgomp1

Las compilaciones de imágenes basadas en distribuciones Python reducidas (como las variantes `-slim`) eliminan bibliotecas compartidas del compilador GNU para ahorrar espacio. No obstante, el motor analítico de **ONNX Runtime** requiere estrictamente del archivo dinámico `libgomp.so.1` para gestionar de forma paralela los hilos de inferencia. La omisión de este paquete resultará en un fallo crítico de tipo `OSError` durante el inicio de la API.

10. Optimización y Resiliencia en Alta Disponibilidad

El sistema implementa patrones avanzados de diseño de software para mitigar cuellos de botella comúnmente asociados con los flujos de inteligencia artificial:

- **Conexiones Reutilizables mediante HTTP Pool:** La clase `httpx.AsyncClient` se instancia con límites fijos configurados en un máximo de 100 conexiones simultáneas concurrentes, lo cual elimina por completo la latencia provocada por el constante proceso de Handshake TCP en redes Cloud.
- **Estabilidad Numérica contra Desbordamientos (Overflow Prevention):** El cálculo matemático de la función Softmax sustrae de manera explícita el valor escalar máximo presente en el tensor analítico antes de computar la función exponencial, blindando al sistema contra errores de tipo `NaN` (Not a Number).
- **Aislamiento de Recursos (Circuit Breaking Implícito):** El establecimiento de un límite estricto de memoria de 1Gi y un parámetro de timeout de red fijado en 5 segundos en las dependencias asíncronas previene el agotamiento en cascada de los recursos informáticos del cluster ante fallas prolongadas de terceros.

11. Solución de Problemas (Troubleshooting)

Fallo en Sincronización Asíncrona (Logs de Red)

Síntoma: Presencia recurrente de la advertencia `Background sync failed: Name or service not known` en el gestor de eventos analíticos del servidor.

Causa Raíz: La variable de entorno `PUNTOS_SERVICE_URL` asignada al contenedor se encuentra mal formulada, apunta a un dominio inexistente o hay un bloqueo estricto en las reglas del Firewall perimetral de la VPC.

Resolución Operacional: Inspeccionar el valor asignado ejecutando comandos de depuración dentro del clúster de Cloud Run y validar la conectividad externa realizando peticiones de prueba directas a través de herramientas de diagnóstico como `curl`.

Predicciones Monotónicas Sesgadas

Síntoma: El modelo evalúa de forma idéntica cualquier estímulo visual asignándole la etiqueta `trash` de manera continua con un índice de confianza extremadamente elevado.

Causa Raíz: Inconsistencia crítica en los tensores de datos provocado por la omisión de los pasos esenciales de estandarización o el uso de factores de escala erróneos (como procesar matrices brutas en el rango entero de 0 a 255 sin conversión flotante previa).

Resolución Operacional: Auditar exhaustivamente la función interna de preprocesamiento de imágenes garantizando la correcta inyección de las constantes estadísticas ImageNet y validar el comportamiento matemático local inyectando un set de datos de prueba balanceado y controlado.

12. Conclusión

El **Servicio de IA** se consolida como un componente tecnológico de ingeniería avanzado, ágil, altamente desacoplado y con tolerancia a fallas. Mediante la articulación balanceada entre la velocidad analítica nativa que confiere el runtime de ejecución ONNX y la arquitectura de eventos no bloqueantes implementada bajo FastAPI, el sistema provee una infraestructura altamente robusta capaz de sustentar la operación automatizada del proyecto EcoSmartBin con estándares de calidad y desempeño óptimos de nivel corporativo.